ADO.NET(Active data objects for .NET)

- **Ado.NET (active data objects for .NET) — .Net-in daxilində olan məlumat bazası ilə əlaqə qurmaq üçün “core” texnologiyadır.**

Yəni:

- **Database-ə qoşulmaq,**
- **SQL sorğularını işlətmək,**
- **Nəticəni oxumaq və obyektlərə çevirmək**

- “Low-level” API dir.
- Çox sürətlidir.
- Amma çox kod yazmaq lazımdır.

```
using System.Data.SqlClient;

string connectionString = "Server=.;Database=TestDb;Trusted_Connection=True;";

using (SqlConnection connection = new SqlConnection(connectionString))
{
    connection.Open();

    string query = "SELECT Id, Name FROM Users";

    using (SqlCommand command = new SqlCommand(query, connection))
    using (SqlDataReader reader = command.ExecuteReader())
    {
        while (reader.Read())
        {
            Console.WriteLine($"{reader["Id"]} - {reader["Name"]}");
        }
    }
}
```

Dapper nədir?

- Dapper-stack overflow tərəfindən hazırlanmış mikro orm-dir.
- ORM(object relational mapper) dedikdə-db nəticələrini avtomatik olaraq c# obyektlərinə map edən kitabxanadır.
- Dapper Ado.Net üzərində işləyir.

Dapper-in məqsədi

- Ado.Net-in eyni performansını saxlayaraq:
- Daha az kod yazmaq
- Nəticələri obyektlərə avtomatik map etmək
- Sadə və yüngül ORM

Necə işləyir?

- Dapper IDbConnection interfeysi ilə işləyir.
- O,Ado.Net-i arxada istifadə edir,sadəcə sən sorğunu sadə formada yazırsan.

```
using Dapper;
using System.Data.SqlClient;

string connectionString = "Server=.;Database=TestDb;Trusted_Connection=True;";

using (var connection = new SqlConnection(connectionString))
{
    var users = connection.Query<User>("SELECT Id, Name FROM Users").ToList();

    foreach (var user in users)
    {
        Console.WriteLine($"{user.Id} - {user.Name}");
    }
}

public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

Ado.Net niyə bu qədər sürətlidir?

- Çünki ado.Net bir başa verilənlər bazası əlaqə saxlayır.
- Serialization yoxdur-bu o deməkdiki,obyekti məlumat formasına çevirib ötürmür.
- Reflection yoxdur-proqramın öz koduna baxa bilmə qabiliyyətidir reflection.Yəni kod runtime da baxa bilir ki bu class ın hansı property si var onun tipi nədir.Aşağıdakı reflectiondır.

Məsələn:

csharp Copy code

```csharp
var props = typeof(User).GetProperties();
```

- ORM lər(entity framework,dapper) bu texnikadan istifadə edir ki,db –dəki sütunları modelə uyğunlaşdırmaq üçün.

ADO.NET-də isə sən bunu özün yazırsan:

csharp Copy code

```csharp
user.Id = (int)reader["Id"];
user.Name = (string)reader["Name"];
```

Yəni **runtime**-da heç bir "öz-özünü analiz" prosesi yoxdur.

⚙️ Nəticə:

- Reflection CPU-ya əlavə yük verir (çünki tip məlumatlarını runtime-da analiz edir)
- ADO.NET bunu etmədiyi üçün **daha yüngül və sürətlidir**

Streaming yanaşma – nə deməkdir?

- “Streaming” dedikdə məlumatın **tam yüklənmədən, hissə-hissə** oxunması nəzərdə tutulur.

Ado.NET –də SqlDataReader belə işləyir:

- Məsələn,db-də 1 milyon sətir varsa,datareader hamısı birdən ram-a yükləmir.
- Əvəzində bir sətir gəlir,sən oxuyursan,sonra növbəti gəlir.

#EF Core və Dapper isə adətən bütün nəticələri RAM-a yükləyir,sonra emal edir.

Connection pooling

- Hər dəfə database-ə qoşulmaq bahalı əməliyyatdır (çünki yeni bağlantı qurulur, autentifikasiya və s. Olur).

Ona görə .Net də connection pooling var:

- ƏGƏR BİR CONNECTİON AÇILIB bağlanıbsa,ado.Net ONU POOL-DA SAXLAYIR.
- Sonrakı açılışda yeni connection yaratmır,əvvəlkini reuse edir.